UNIVERSIDAD DEL PACÍFICO



Sesión 9

Tratamiento de valores faltantes y codificación de variables

Docente: Soledad Espezua Llerena

s.espezual@up.edu.pe

Objetivo de la sesión

Transformar una base integrada sin perder su significado: primero resolvemos los faltantes y luego representamos correctamente las variables categóricas.

Proceso hacia una matriz analítica

Base original

Pacientes · Atenciones · Medicación

Base limpia e integrada

Una fila por unidad de análisis

Transformación

Codificar · derivar · discretizar

Matriz analítica

Variables listas para aplicar técnicas de minería

- ✓ Diagnóstico de calidad
- ✓ Limpieza estructural
- ✓ Integración de tablas
- ✓ Unidad de análisis
- ✓ Exploración y visualización

- ○ Decidir qué hacer con faltantes
- ○ Eliminar o imputar cuando corresponda
- ○ Codificar variables categóricas
- ○ Construir una primera representación de matriz analítica

Matriz analítica

Es una tabla organizada para realizar análisis posteriores sobre una unidad de análisis definida pero cambia la representación de algunas variables.

- Cada fila representa una unidad de análisis (por ejemplo, un paciente).
- Cada columna representa una característica o variable relevante.
- Las categorías se representan de forma adecuada: binaria, ordinal o one-hot.
- Puede incluir variables derivadas a partir de otras fuentes o cálculos.
- Los identificadores (id_paciente) se conservan como referencia, pero no representan una característica analítica.

Ejemplo: una fila = un paciente

id_paciente	edad	asegurado_bin	n_atenciones	ult_atencion_Control
P001	64	1	2	1
P002	45	0	1	0
P003	70	1	2	1

El id_paciente se conserva solo para trazabilidad pero no se incluye como variable analítica porque no representa una característica del paciente.

Valores faltantes



Tratamiento de valores faltantes

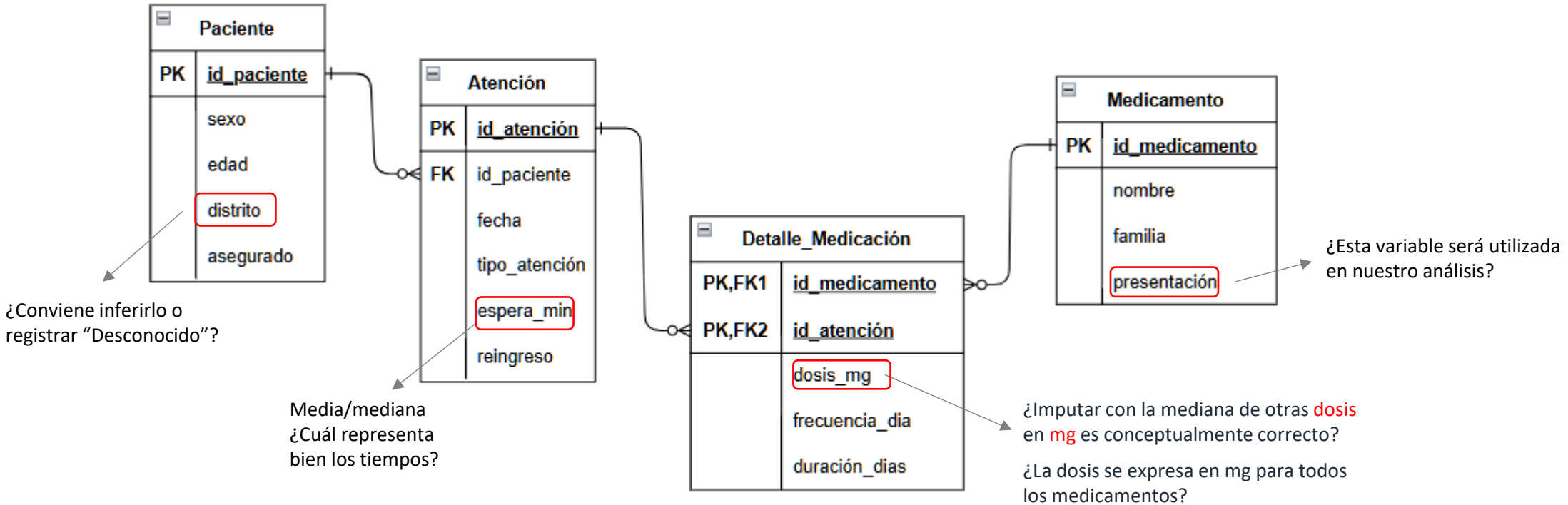
Revisar si la variable es necesaria→ Evaluar si conviene eliminar→ Elegir una imputación razonable cuando corresponda→ Verificar qué cambió después del tratamiento

Situación	Posible decisión
Pocas filas faltantes y su pérdida es aceptable →	Eliminar filas
Variable poco útil o no necesaria para el análisis →	Excluir variable
Variable numérica donde el faltante representa un dato que debería existir →	Imputar media o mediana
Variable categórica nominal →	Sustituir con la Moda o crear categoría “Desconocido”
Faltante con significado especial →	No imputar sin revisar

Eliminar, imputar o dejar el faltante

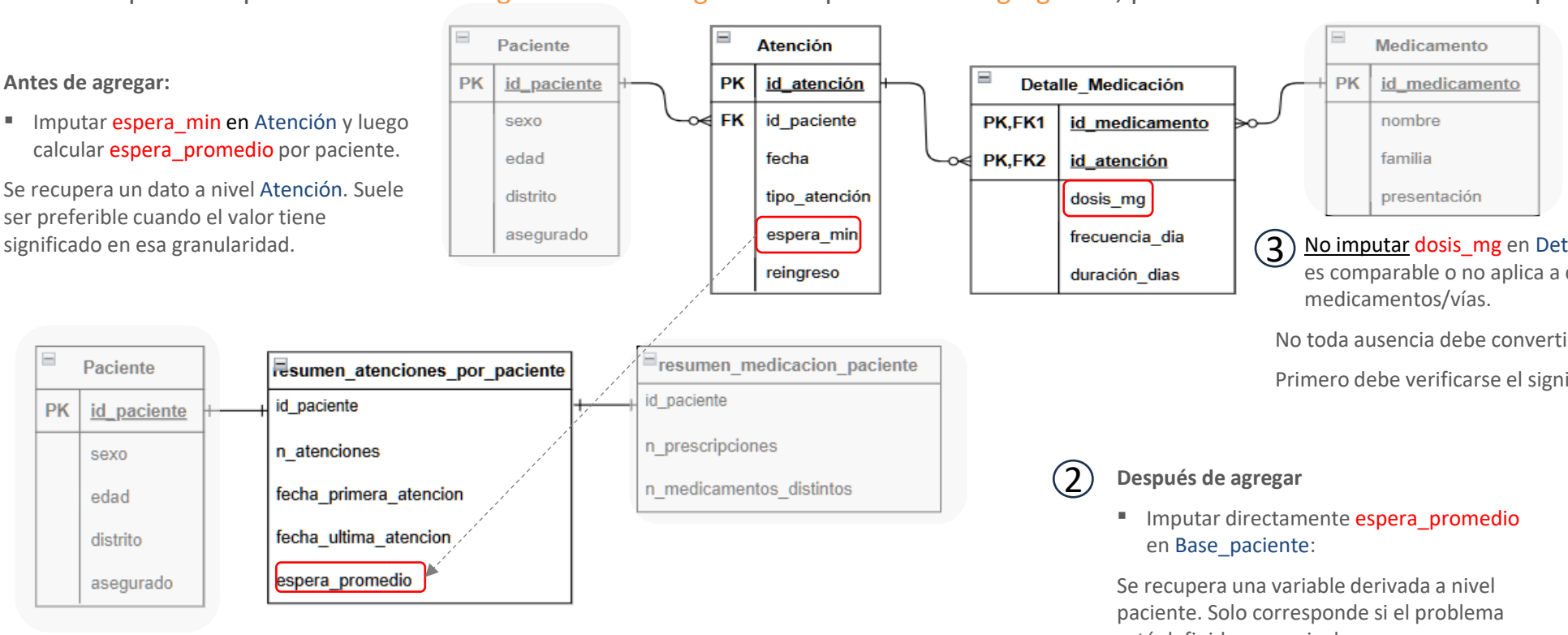
Caso clínica

Primero entendemos el contexto del faltante. Después elegimos el tratamiento.



¿En qué nivel conviene imputar?

La imputación puede hacerse en la granularidad original o después de una agregación, pero las dos decisiones no son equivalentes.



① **Antes de agregar:**

- Imputar espera_min en Atención y luego calcular espera_promedio por paciente.

Se recupera un dato a nivel Atención. Suele ser preferible cuando el valor tiene significado en esa granularidad.

③ No imputar dosis_mg en Detalle cuando la medida no es comparable o no aplica a ciertos medicamentos/vías.

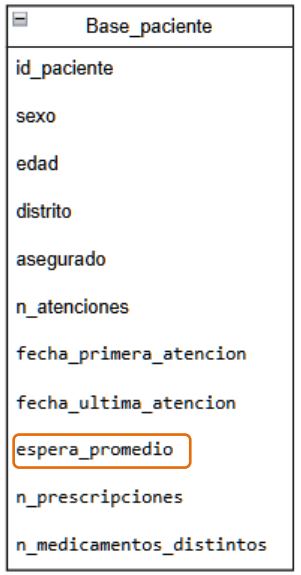
No toda ausencia debe convertirse en un número.

Primero debe verificarse el significado de la variable.

② **Después de agregar**

- Imputar directamente espera_promedio en Base_paciente:

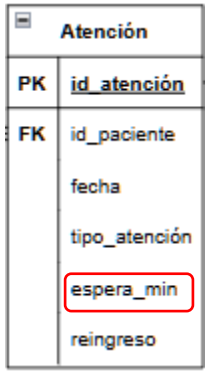
Se recupera una variable derivada a nivel paciente. Solo corresponde si el problema está definido a ese nivel.



Hay que imputar en la granularidad donde el valor tiene significado: cuando sea conceptualmente válido.

Ejemplo 1: imputación de una variable numérica

Caso A: imputar antes de agregar



Imputar una variable numérica espera_min.

Este faltante está en Atención→ lo tratamos a nivel Atención y luego calculamos espera_promedio por paciente.

```
# 1. Revisar la variable original a nivel atención
atenciones["espera_min"].describe()

# 2. Imputar con la mediana
mediana = atenciones["espera_min"].median()
atenciones["espera_min_imp"] = (
    atenciones["espera_min"].fillna(mediana)
)

# 3. Agregar a nivel paciente
resumen_atenciones = (
    atenciones.groupby("id_paciente")
    .agg(espera_promedio=("espera_min_imp", "mean"))
    .reset_index()
)
```

1) Analizar la distribución

2) Imputar

Crear una variable nueva permite comparar

¿Por qué mediana?

- Menos sensible a valores extremos.
- Puede representar mejor el valor central cuando la distribución es asimétrica.

¿Por qué antes de agrupar?

- El faltante corresponde a una atención.

3) El **promedio** se calcula después con los datos ya tratados.

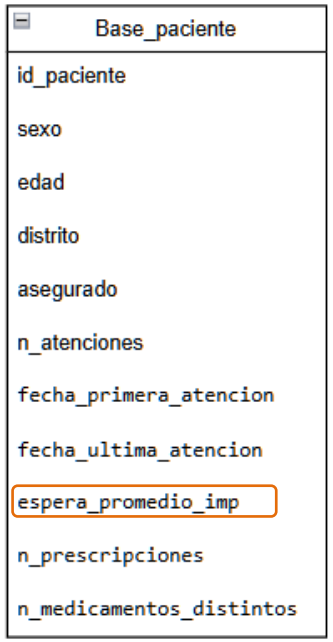
Media y mediana son opciones; la elección depende de la distribución y del contexto.



Tener cuidado cuando el porcentaje de faltantes es alto porque imputar altera la distribución de los datos

Ejemplo 1: imputación de una variable numérica

Caso B: imputar después de agregar



El faltante se trata en la base analítica, cuya unidad de análisis ya es el paciente → lo tratamos a nivel Base_paciente.

espera_min —agrupar→ espera_promedio —integrar base_paciente→ **espera_promedio_imp**

1) Analizar la distribución

```
# 1. Revisar la variable a nivel paciente
base_paciente["espera_promedio"].describe()

# 2. Imputar con la mediana
mediana = base_paciente["espera_promedio"].median()

base_paciente["espera_promedio_imp"] = (
    base_paciente["espera_promedio"].fillna(mediana)
)
```

Crear una variable nueva para comparar

2) Imputar con la **mediana**

¿Cuándo tiene sentido?

- Ya estamos trabajando a nivel paciente y espera_promedio es una variable de la matriz analítica.
- Queremos conservar todos los pacientes y necesitamos utilizar el tiempo promedio de espera.

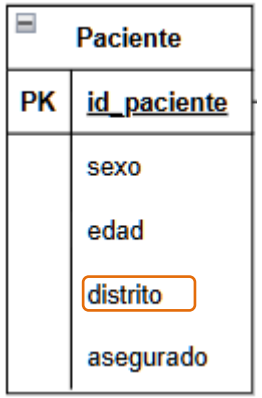
Atención

groupby().mean() ignora NaN

Ejemplo: 20, NaN, 40 → espera_promedio = 30.

En este caso imputar después de agregar no recupera esa atención faltante.

Ejemplo 2: faltante en una variable categórica



Para distrito, dos tratamientos simples producen significados distintos:

Opción A

Selecciona la primera moda

```
# Opción A: imputar con la moda
moda = base_paciente["distrito"].mode()[0]

base_paciente["distrito_moda"] = (
    base_paciente["distrito"].fillna(moda)
)
```

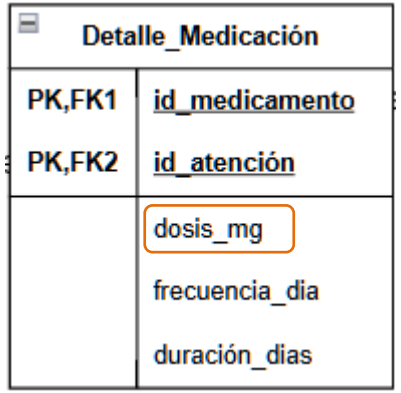
Opción B

```
# Opción B: conservar que no conocemos el distrito
base_paciente["distrito_imp"] = (
    base_paciente["distrito"]
    .fillna("Desconocido")
)
```

	Ventaja	Riesgo
Moda	Mantiene categorías existentes	Puede inventar una categoría sin evidencia
Desconocido	Hace visible la ausencia	Crea una categoría adicional "Desconocido"

Ejemplo 3: no todo faltante debe imputarse

Una imputación numéricamente válida puede ser semánticamente incorrecta.



El faltante aparece en una prescripción: dosis_mg → variable numérica

Sin embargo, antes de imputar debemos verificar si la variable y su unidad son comparables entre medicamentos y presentaciones.

Por ejemplo, una **dosis** puede estar expresada en **mg**, mientras que otra presentación puede utilizar una unidad diferente.

Pregunta antes de imputar

- ¿La variable usa la misma unidad en todos los registros?
- ¿El faltante significa "no disponible" o "no aplica"?
- ¿Necesitamos esta variable para el análisis actual?

Imputar es una decisión analítica, no una obligación.

```
# Mostrar registros con dosis faltante
detalle[
    detalle["dosis_mg"].isna()
]
```

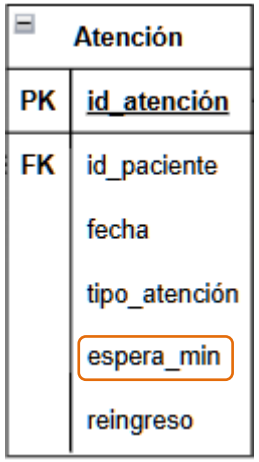
Detectamos el faltante, revisamos su contexto y decidimos:

- **no imputar** porque no podemos garantizar que los valores sean comparables.
- **excluir** dosis_mg de la matriz analítica actual si esta variable no es necesaria para el análisis actual.

Tratamiento mediante eliminación: tres casos diferentes

Caso A: eliminar en una granularidad más fina, antes de agregar

El faltante está en una tabla de granularidad más fina: **una fila = una atención.**



Eliminar filas: atenciones sin tiempo de espera

```
# Cuántas atenciones tenemos antes
n_antes = atenciones.shape[0]

# Eliminar filas con faltante en una variable
atenciones_limpias = atenciones.dropna(subset=["espera_min"])

n_despues = atenciones_limpias.shape[0]
print("Atenciones eliminadas:", n_antes - n_despues)
```

Conviene cuantificar cuántas filas se pierden al aplicar la eliminación.

Use esta opción cuando la fila incompleta no sea utilizable y la pérdida de registros sea aceptable.

¿Qué se pierde?

Se eliminan **atenciones**. El paciente puede permanecer si conserva otras atenciones válidas.

Eliminar en una granularidad fina, modifica los datos que después se usarán para construir los resúmenes.

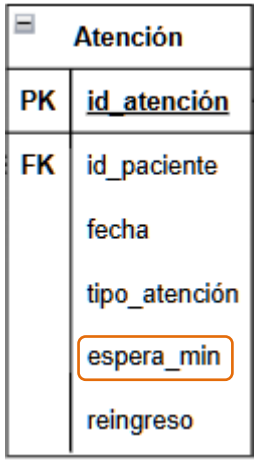
Por ejemplo, si un paciente tenía tres atenciones y eliminas una, su espera_promedio se calcula solo con las dos restantes.

Luego se agrega:

```
resumen_atenciones = (
    atenciones_limpias
    .groupby("id_paciente")
    .agg(espera_promedio=("espera_min", "mean"))
    .reset_index()
)
```

se calcula espera_promedio por paciente

Tratamiento mediante eliminación: tres casos diferentes




```
# Eliminar filas con faltantes
df.dropna(
  subset=[...],
  how="any"
)
```

Parámetros dropna

`subset=[...]`	*Columnas en las que se buscarán faltantes.*
`how="any"`	*Elimina si falta al menos una de las columnas consideradas.*
`how="all"`	*Elimina solo si faltan todas las columnas consideradas.*
`axis=0`	*0: elimina filas.*

¿Por qué faltan datos en espera_min?

Eliminar faltantes con dropna() no solo reduce el número de filas. También puede cambiar la composición de la muestra si la ausencia no ocurre al azar.

- **MCAR**: la ausencia de espera_min ocurre por azar y no depende ni de otras variables ni del propio valor faltante.

  Ejm. Un error del sistema impidió guardar espera_min en algunas atenciones (puro accidente)

- **MAR**: la ausencia de espera_min se relaciona con otra variable observada.

  Ejm. espera_min falta más frecuentemente en atenciones de Emergencia, porque allí el registro se realiza de otra forma.

- **MNAR**: la ausencia de espera_min se relaciona con el propio valor faltante o con información no observada.

  Ejm. Los tiempos de espera_min muy largos tienen mayor probabilidad de no registrarse.

Si eliminamos esas filas…

→ Se reduce el número de observaciones, pero la muestra no debería cambiar sistemáticamente.

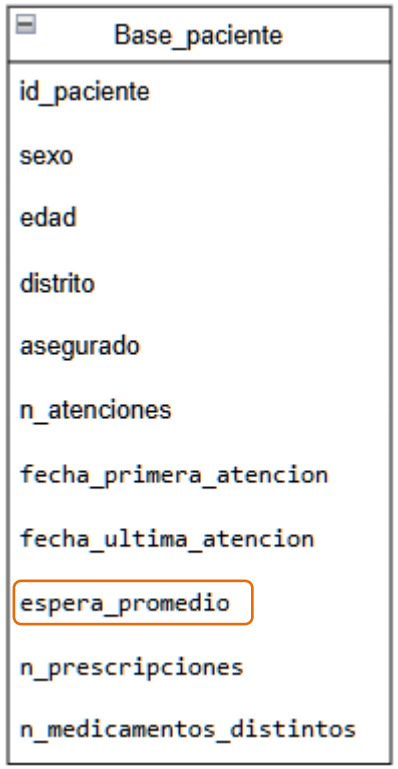
→ Podríamos eliminar más atenciones del tipo Emergencia y alterar la composición de la muestra.

→ Podríamos subestimar la espera real, porque tenderían a quedar principalmente tiempos menores.

Tratamiento mediante eliminación: tres casos diferentes

Caso B: eliminar en la tabla analítica final

La unidad de análisis ya es paciente:
una fila = un paciente.



Eliminar filas: pacientes con espera_promedio faltante

número de pacientes

```
n_antes = base_paciente.shape[0]

base_reducida = base_paciente.dropna(subset=["espera_promedio"])

n_despues = base_reducida.shape[0]
n_eliminados = n_antes - n_despues
porcentaje = n_eliminados / n_antes * 100

print("Pacientes eliminados:", n_eliminados)
print(f"Porcentaje eliminado: {porcentaje:.1f}%")
```

Antes de eliminar observaciones de la matriz analítica, cuantifique **cuántas unidades de análisis** se perderán y qué **porcentaje** representan.

En la tabla analítica final,
una fila eliminada = una unidad de análisis menos.

¿Qué se pierde?

Se eliminan **pacientes**.

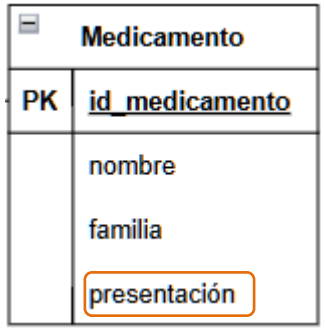
```
base_paciente.dropna()
```

sin subset es bastante agresivo, porque por defecto elimina toda fila que tenga al menos un NaN en cualquier columna.

Tratamiento mediante eliminación: tres casos diferentes

Caso C: excluir una variable que no será utilizada

Si una variable tiene **muchos faltantes** y además **no es necesaria** para el objetivo del análisis, puede decidirse excluirla.




```
# Eliminar una columna que no será utilizada
df = df.drop(
    columns=["presentacion"]
)
```

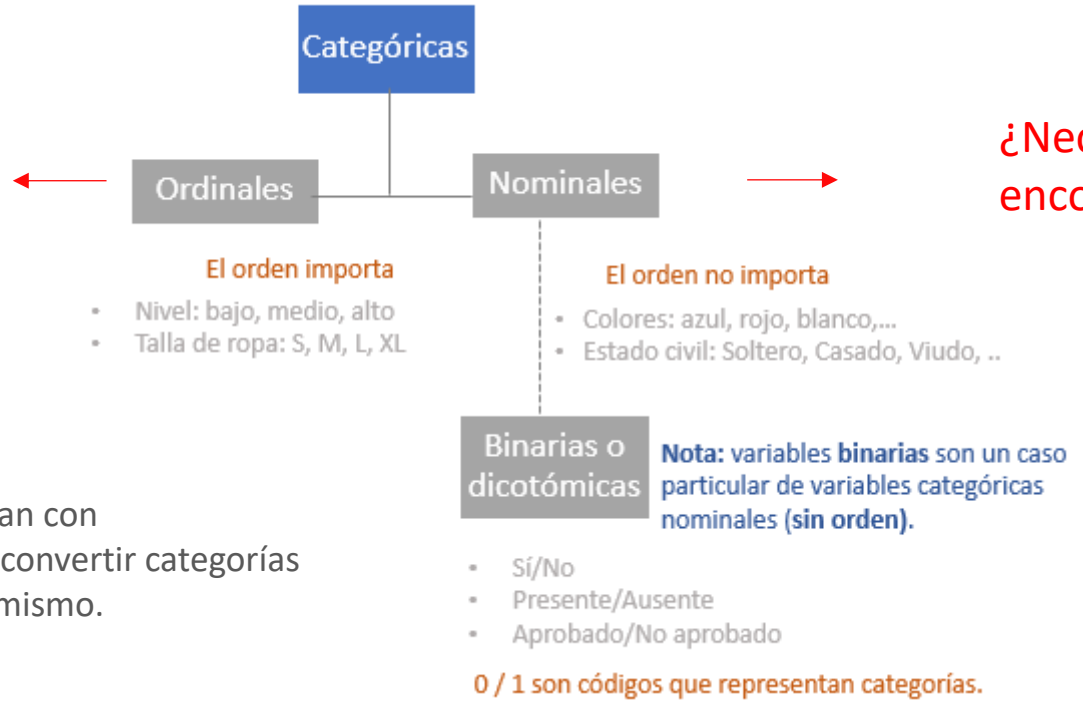
Pasos para tratar faltantes

1. Detectar	→ ¿Dónde están los faltantes y cuántos son?
2. Comprender	→¿Por qué podrían faltar? ¿MCAR, MAR, MNAR, no aplica?
3. Evaluar utilidad	→¿La variable es necesaria para el análisis actual?
4. Elegir tratamiento	→¿Eliminar, imputar o conservar como desconocido?
5. Aplicar	→¿Qué código reproduce exactamente la decisión?
6. Verificar	→¿Quedaron faltantes? ¿Cambió la distribución o la cantidad de casos?

Primero se toma una decisión analítica; después se realiza el código.

Después de tratar los faltantes: representar variables categóricas

¿Existe realmente un orden entre categorías?



¿Necesita one-hot encoding?

Muchas técnicas de minería trabajan con representaciones numéricas. Pero convertir categorías a números no siempre significa lo mismo.

Variables categóricas

El código numérico debe respetar el significado de la variable.

Tipo de variable	Escala	Orden	Distancias comparables	Ejemplo	Representación posible
Categórica	**Nominal**	No	No	tipo_atencion: Consulta / Control / Emergencia	One-hot encoding
Categórica	**Ordinal**	Sí	No	prioridad: Baja / Media / Alta	1 / 2 / 3, si el orden es real
Categórica	**Binaria**	No	No	asegurado: Sí / No	0 / 1

Codificación nominal: One-hot

tipo_atencion	Código incorrecto
Consulta	1
Control	2
Emergencia	3

¿Por qué no usar 1, 2, 3?

Ese código introduce artificialmente orden y distancia: parecería que Emergencia > Control > Consulta.

One-hot encoding con `pd.get_dummies()` es apropiado cuando las categorías no tienen un orden natural.

```
base_transformada = pd.get_dummies(
    base_paciente,
    columns=["tipo_ultima_atencion"],
    prefix="ult_atencion",
    dtype=int
)
```

`columns=[...]` reemplaza la columna nominal por las nuevas columnas indicadoras.

Una variable se transforma en varias columnas indicadoras

tipo_ultima_atencion	ult_atencion_Consulta	ult_atencion_Control	ult_atencion_Emergencia
Consulta	1	0	0
Control	0	1	0
Emergencia	0	0	1

La columna original puede conservarse en una base descriptiva separada.

One-hot no se recomienda para variables que tengan muchos niveles de categoría

Codificación ordinal con `map()`

Cuando existe un orden real no es necesario inventar un orden.

```
orden = {
    "Baja": 1,
    "Media": 2,
    "Alta": 3
}

df["prioridad_ord"] = (df["prioridad"].map(orden))
```

Baja < Media < Alta, existe un orden natural

prioridad	Código
Baja	1
Media	2
Alta	3

Codificación binaria con `map()`

Para dos categorías, map() permite definir explícitamente qué valor corresponde a cada una.

0 y 1 son códigos de categorías. No significan que una categoría sea “mayor” que la otra.

```
base_paciente["sexo_bin"] = ( base_paciente["sexo"].map({"F": 0, "M": 1}))

base_paciente["asegurado_bin"] = (base_paciente["asegurado"].map({"No": 0, "Sí": 1}))
```

Variable	Categorías	Nueva variable
sexo	F / M	sexo_bin
asegurado	No / Sí	asegurado_bin
reingreso	No / Sí	reingreso_bin

También puede aplicarse a variables derivadas, por ejemplo: “tuvo algún reingreso” o “recibió un antibiótico”.

Decisión posterior: ¿mantengo también la columna textual original?
Depende del propósito de la base.

Primera matriz analítica

La matriz analítica es una representación preparada para las técnicas posteriores de minería de datos.

Variable original	Decisión	Representación
id_paciente	Conservar solo como identificador	Excluir como característica de la matriz analítica
edad	Conservar	Numérica
sexo	Codificar	sexo_bin
asegurado	Codificar	asegurado_bin
espera_promedio	Tratar faltante	espera_promedio_imp
tipo_ultima_atencion	Codificar	Columnas one-hot
distrito	Según objetivo	Imputar/Desconocido y eventualmente codificar

UNIVERSIDAD DEL PACÍFICO

¡Gracias por su atención!